

Advanced Computer Networks



Dr Sudipta Saha

Associate Professor

Dept of Computer Science & Engineering
Indian Institute of Technology Bhubaneswar

Congestion control in TCP



DSSRG:
Decentralized Smart
Systems Research Group

<https://sites.google.com/iitbbs.ac.in/dssrg>

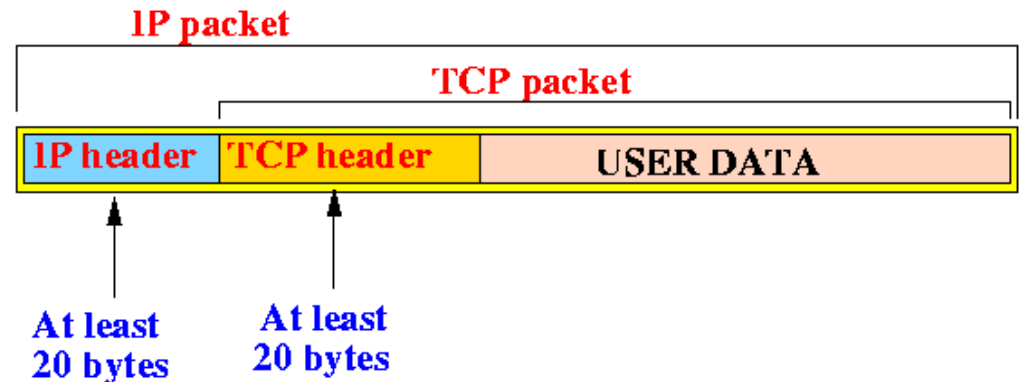
Or Google *dssrg iitbbs*



The Congestion Control Algorithm in TCP

Each packet carries a **single byte of data**, the IP packet size will be **41 bytes**.

How user data is transmitted on the Internet:



The efficiency (useful part of the packet) would be **1/41** or **2.4%**

That's equivalent to 97.6% overhead !!!

TCP will always try to send **multiple bytes in a packet** to improve efficient

Assumptions

- Let us assume that the **TCP sources** are transmitting ***continuously***
- Furthermore, a **TCP source** will **always** send the **largest TCP segment** (for efficiency reasons)
- **TCP** always transmits packets of **size** equal to **MSS** bytes
- **Maximum Segment Size (MSS)**: The system can impose a maximum packet (segment) size on **TCP packets**



Getting to Equilibrium: Slow-start

- Fine when we reach the equilibrium.
- **But** How to reach the equilibrium ?? (Either in starting or restarting after a packet loss)
- We need a clock... which will tell the sender to act – transmit or stop
- **Observation:**
 - “*Receiver cant generate the ACKs no sooner than the data packets can get through the network*”



The principle behind

- All problems starts from local buffer overflows
- There is no issue in TCP protocol as such!.....
- It all depends on how the TCP protocol is implemented over the network
- **Windowing mechanism – may produce wrong behavior and correct behavior – based on the situation**
- The primary principle followed is the principle of “**Packet conservation**”

The transport connections are forced to obey this principle of “Packet conservation”



“Conservation of packet”

- A connection ‘in equilibrium’ →

Running stably with a full window of data in transit

- The packet flow is ‘conservative’ →

A new packet isn’t put into the network until an old packet leaves

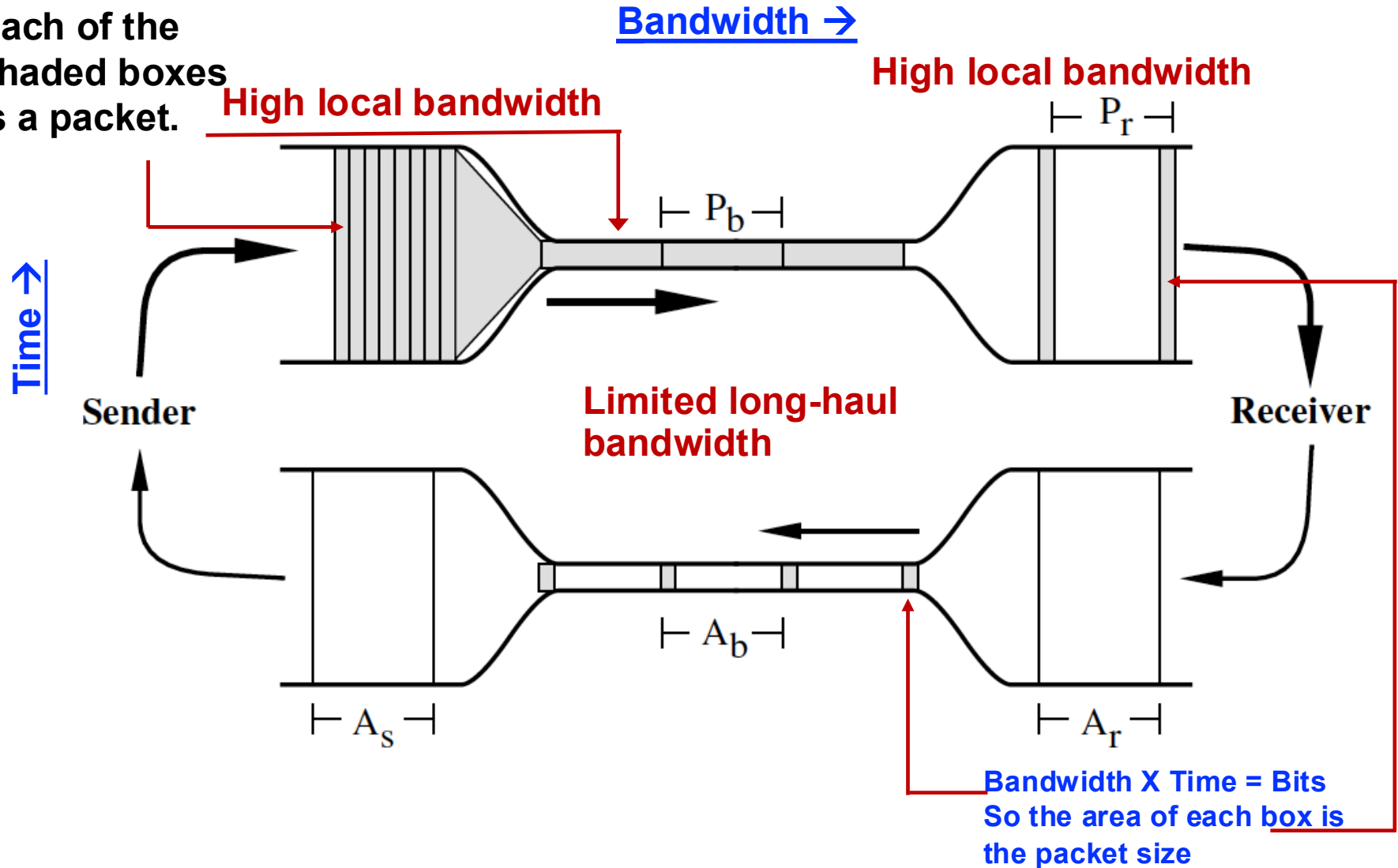
- A conservative flow → *For any given time, the integral of the Packet density around the sender–receiver–sender loop is a constant.*

Aim: **Congestion collapse to be an exception not a rule**

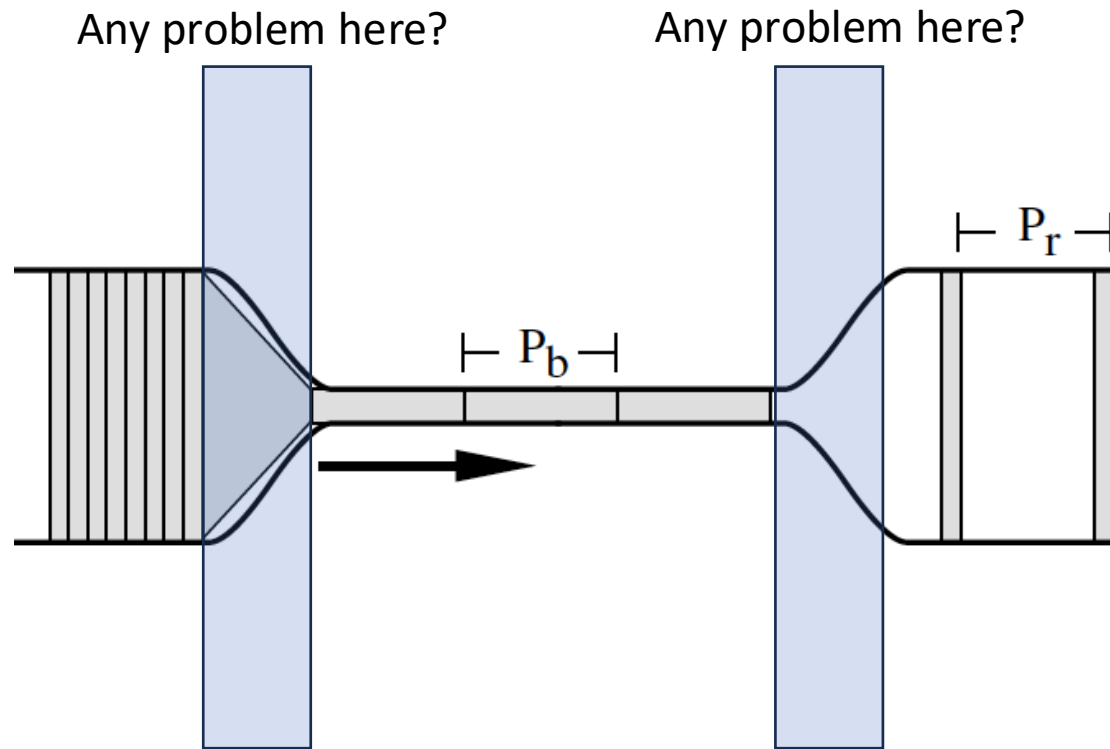
Congestion control: **Fix where it does not obey**



Each of the shaded boxes is a packet.



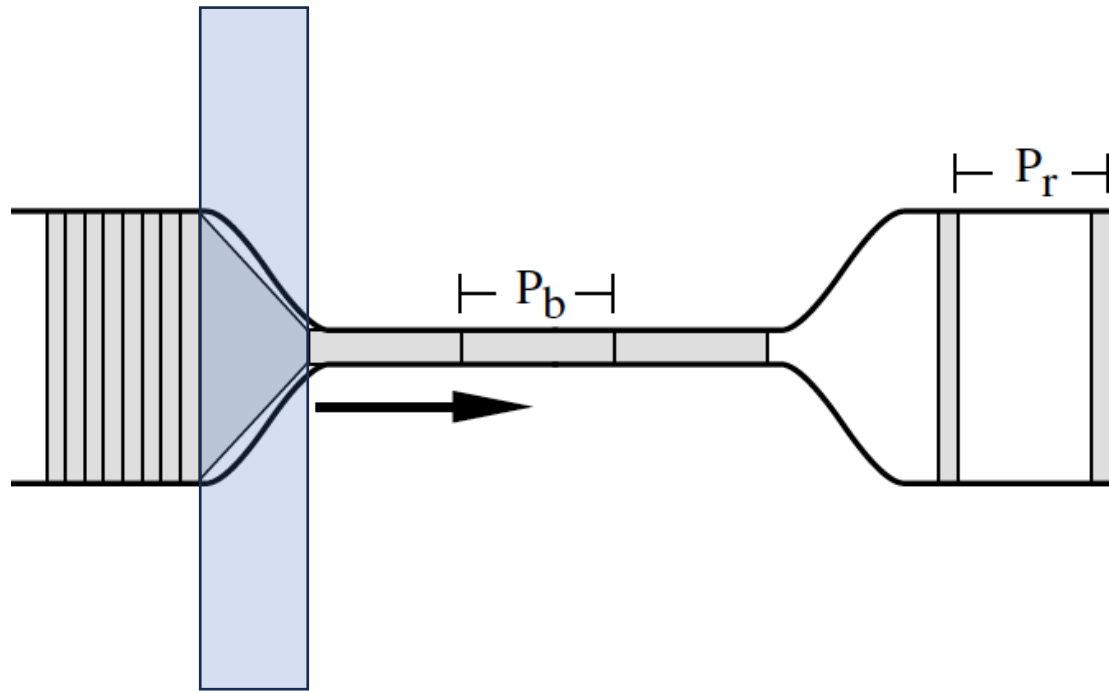
- The number of bits doesn't change as a packet goes through the network
- A packet gets squeezed into the smaller long-haul bandwidth must spread out in time.



If sender transmits at higher rate

And this slow link does not have enough buffer then what?

It will start packets to get dropped



Sender does not know the buffer size allocated here

So better to start with one and send when you receive the ACK

Slowly increase...

The time **P_b = Minimum packet spacing on the slowest link** in the path (the bottleneck).

After bottleneck the inter-packet interval does not change

At Receiver packet spacing **$P_r = P_b$**

If the receiver processing time is the same for all packets, the spacing between ACKs on the receiver side $\rightarrow$ **$A_r = P_r = P_b$**

If the time slot P_b was big enough for a packet, it's big enough for an ACK, spacing is preserved along the return path.

Ack spacing at sender **$A_s = P_b$** .

Conclusion: If packets are sent only in response to an ACK, the sender's packet spacing will exactly match the packet time on the slowest link in the path.



Self-clocking

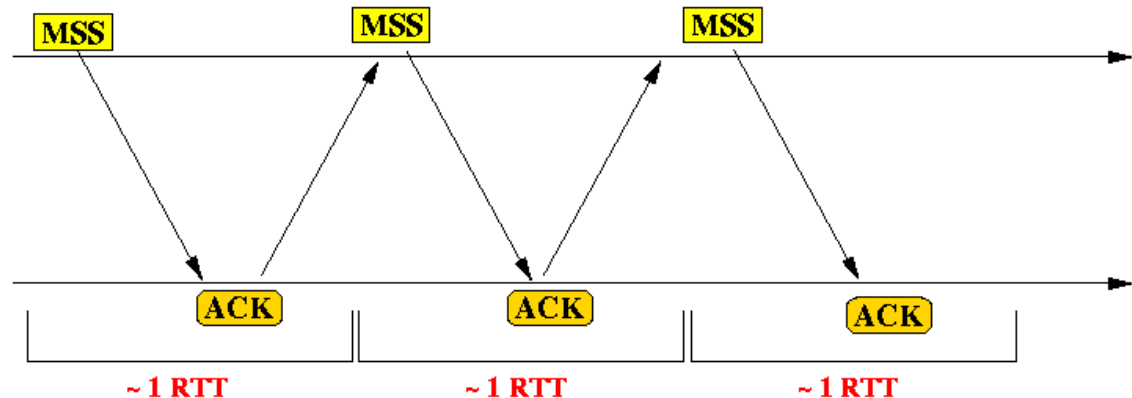
- This is known as self-clocking
- So its best to take decision when you receive an ACK
- But still how to reach the equilibrium?
- Slowly increase the number of packets transmitted together as an ACK is received → ***Gradually increase data in transit***
- ***CWND per connection, Set to 1***



Window size and Transmission Data Rate

- The **transmission data rate** is dependent on the **transmit window size**
- **Window size** will **limit the number of packets** that the **sender** can **transmit continuously**
- **Example 1: Window size** is equal to **MSS** (Max Segment Size)

win = MSS



$$\text{Transmission Rate} \approx \frac{\text{MSS}}{\text{RTT}} \text{ bps}$$



Explanation

Transmit window size is equal to ***MSS***

Sender can **send only 1 packet at a time** and must **pause** (wait for window to slide)

The **ACK** for the packet will **be received** in **approximately RTT (round trip time)** sec

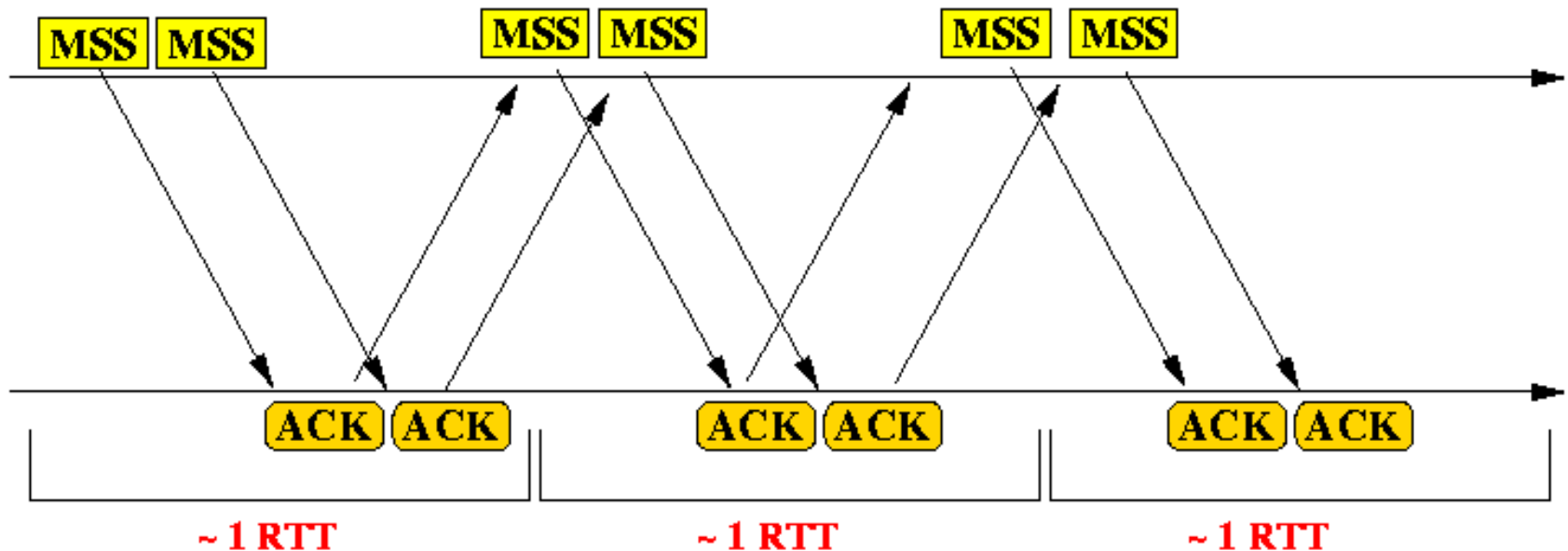
When the **ACK is received**, the sender **sends the next packet**

The **resulting transmission (data) rate** is approximately **MSS/RTT bps.**



Example 2: Window size = $2 \times MSS$, the sender can send faster

win = $2 \times MSS$



$$\text{Transmission Rate} \sim = \frac{2 \times MSS}{RTT} \text{ bps}$$

Slow-start

- **Slow-start window increase isn't that slow: It takes time $R \log_2 W$ to reach W**

R = round-trip-time and W = window size in packets

- The window opens quickly enough to have **a negligible effect on performance**, even on links with a large bandwidth–delay product.
- The algorithm guarantees that a connection will source data at a rate **at most twice the maximum possible on the path**. [*Opening a window of size W packets will require $W/2$ packets of buffer capacity at the bottleneck*]
- Example: 10 Mbps Ethernet hosts talk over the 56 Kbps → The gateways will see a burst of packets delivered at 200 times the path bandwidth.
- This burst of packets often puts the connection into a persistent failure mode of continuous retransmissions



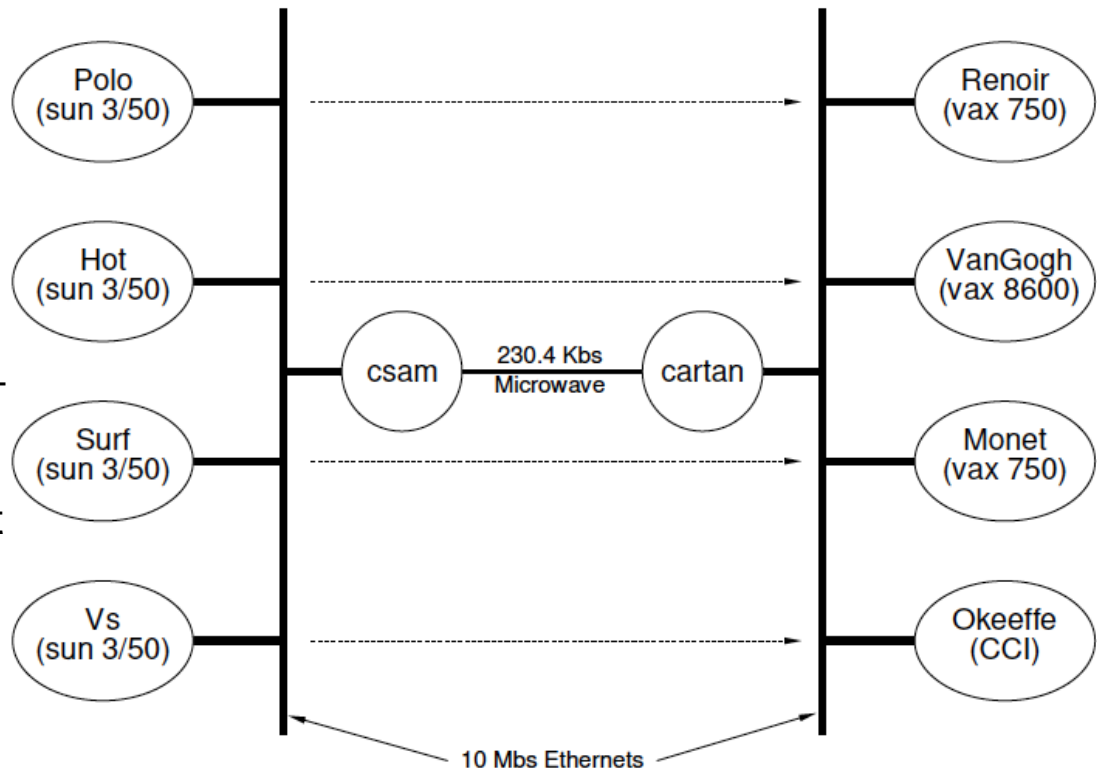
Test setup - TCP Performance with a bottleneck link

LBL (Lawrence Berkley National Laboratory) $\leftrightarrow$ UCR (University California Riverside) – **about** - 400 miles around

Two IP Routers – 4 connections :
230 Kbps Link whereas - Local network faster in Mbps

4 connections: 1 Mbytes transfer –
2048 no of 512 Byte data packets
were initiated 3 seconds apart from
4 machines at LBL to 4 machines at
UCR

There are 6 store and forward hops



Each connection:

Has a window of 16 KB i.e., 32 no of 512 Byte packets

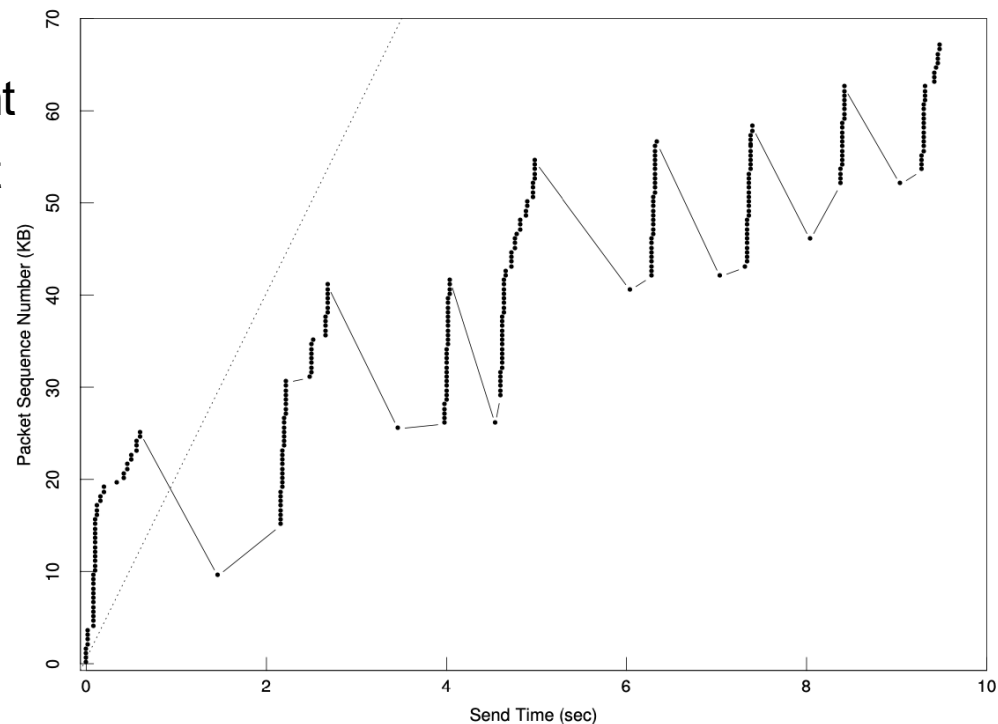
The microwave link can queue upto 50 packets. $32 \times 4 = 128$ packets total

All four senders may send – which will overflow the queue upto about 160%



Behavior without Slow-Start

- Each dot: **A 512 Byte packet**
- **x-axis** – Time when packet was sent
- **y-axis** -- Sequence no of the packet
- **Vertical array of dots** –
back to back transmissions
- Two dots having same sequence no
→ **Retransmissions**
- *Smooth line* → Desirable behavior
- **Slope** →
Available bandwidth (20 KBps)



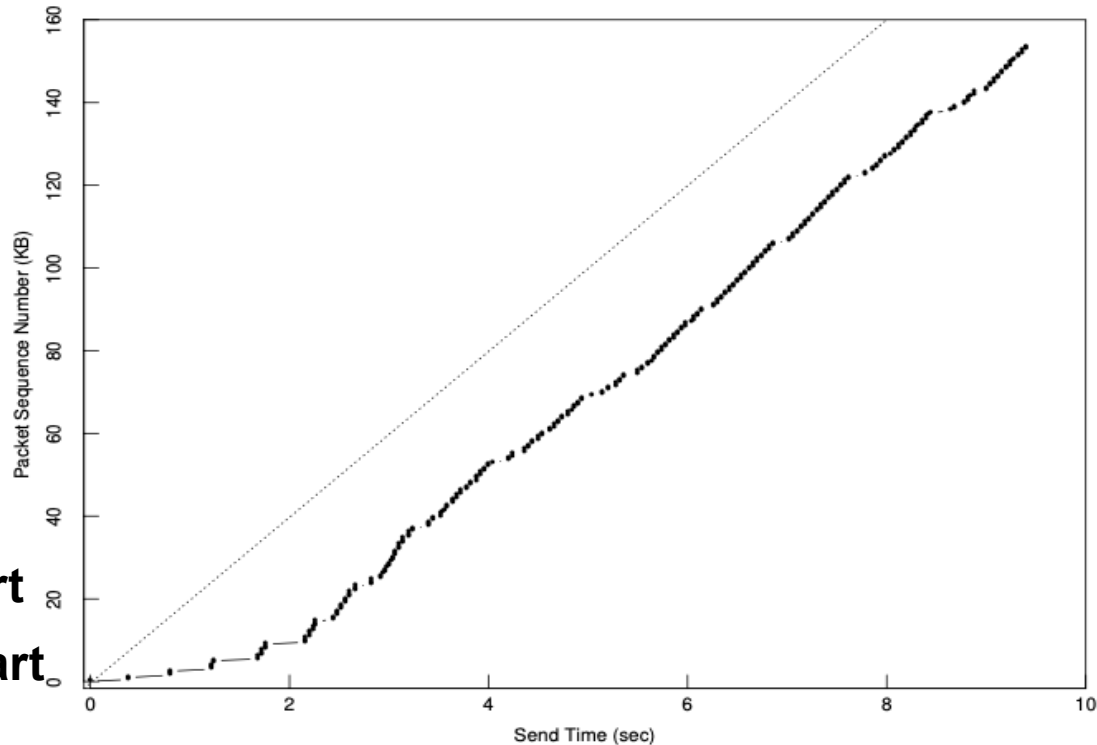
- We get is only 35%
- Rest is wasted on Retransmits
- **Everything is retransmitted at least once**

Data from 54 to 58 KB is sent 5 times



Behavior with Slow-Start

- Same buffer, Same window size, Same network paths
- But now it uses **Slow-Start**
- No bandwidth is wasted on **Retransmits**
- But 2-sec is spent on **slow-start**
- Effective bandwidth of **slow-start part is less (16 KBps)**
- But total effective bandwidth is **20 KBps**
- In earlier case the effective bandwidth has been 7 KBps



TCP's Congestion Window Size

- The transmit data rate *increases* approximately *linearly* with the window size [So far...]
- Congestion will occur at the output link that receives the **most amount of data** and/or **has the lowest bandwidth**
- Above a certain transmission rate the output buffer at the **"weakest" output link will overflow**

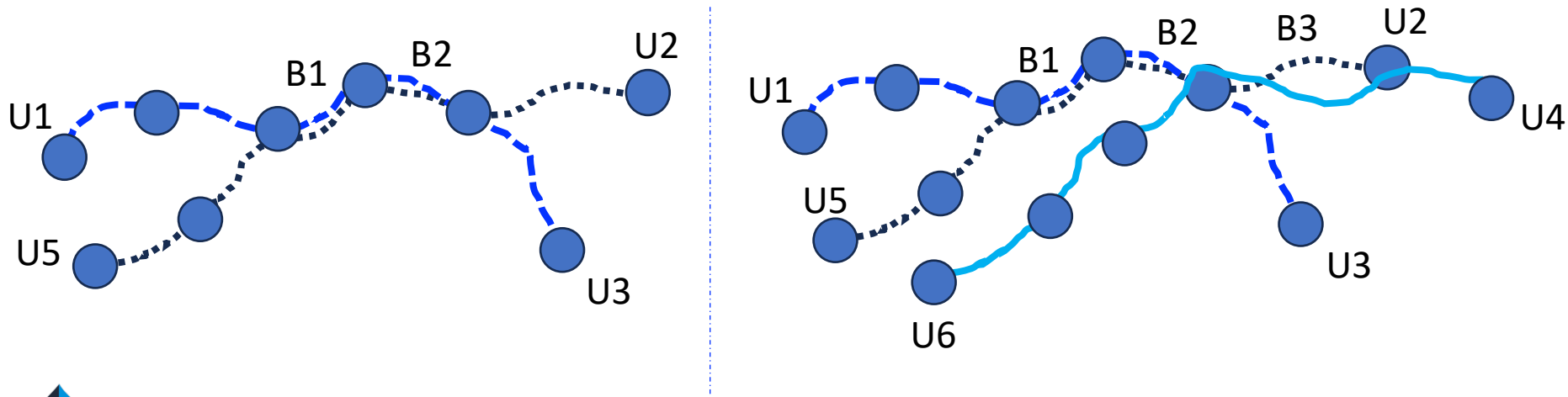
Thus – We need to introduce CONGESTION WINDOW

- **Congestion Window Size** = the number of packets (of MSS bytes of data each) that the sender *believes* that it can transmit into the network *without causing congestion* in the network.



Congestion window size

- The **congestion window size** *cannot* be accurately determined
- The **congestion window size** *varies* (changes) over time because it depends on the **current network status**
- [Path may also change, so weakest link may also change]



Advertised Window Size

- **Advertised Window Size = number of bytes that the receiver is willing to buffer (Used for flow-control recall)**
- **The sender cannot send more than the *advertised window size***
- Different windows - so far we have seen –

1. Transmission Window Size

[Based on Reliability Protocol] – to be determined

2. Congestion Window Size [Congestion in Network]

3. Advertised Window Size [Flow Control issue]



Relationship

- ***Transmit Window* = # packets** (of **MSS** bytes in each packet) that **sender** can **transmit without having to wait for acknowledgement**
- **We call it send window**
- The **size** of the ***Transmit* Window** is **computed (determined)** as the **minimum** of these **2 windows**:
- **Transmit window size =**
min (Advertised Window Size ,
TCP's Congestion Window Size)



Notations used in TCP congestion control scheme

- **Advertised Window Size (AWS)** = amount of data that the receiver is willing to buffer.
- **AWS** is determined by the available space in the receiver buffer
- **Congestion Window Size (CWND)** = window size *imposed* on TCP by the current network state to avoid congestion (buffer overflow)
- **CWND** depends on network traffic
- **Transmit Window Size (TWS)** = the transmit (send) window used by the sender
- It is the amount of *unacknowledged* data that the sender can have outstanding. i.e., the amount of data that TCP can transmit without receiving ACKs
- **$TWS = \min(AWS, CWND)$**



Assumption in TCP congestion control

- If **AWS < CWND**, then the **sender** will **always** use a window size **AWS**
- **Furthermore**, it will **not** cause congestion in the network (Because **AWS < CWND**...)
- If the **sender** does *not* cause congestion, then the **congestion control** problem is *not there* ...
- So we assume $\rightarrow$ **AWS $\geq$ CWND**



TCP congestion control

Analogy with video game playing -

1. Play **as fast as you can**, until **you get killed** / get defeated / Fail
2. Remember *where* and how failed
3. Restart the game from the *beginning*, and play *very quickly before* the point where failed
4. From that point on, proceed *very carefully*
5. Continue to play *carefully* until fail and repeat from step 3



Level 1 to 16:
Quickly cover and
then play more carefully



Same is followed -

- **TCP detects congestion** (through a *packet loss*) and sets the **Congestion window (CWND)**
- **TCP sets a variable SSThresh (Slow Start Threshold) to:**
SSThresh = CWND/2 [Why ???]
- **Rationale :**
- **Half of the Congestion Window size is a conservative estimate will hopefully *not* cause congestion** even if there is a bit of change in the dynamics



Contd...

- **TCP** will restart by transmitting using **CWND = 1**, So TWS is also 1, i.e., ONE packet outstanding
- **Increase CWND exponentially** (from 1) to **SSThresh** → This phase of TCP is called the **slow start phase**
- When **CWND $\geq$ SSThresh**, (this is the point where **TCP believes it is still safe**), it will enter the **careful progress phase**
- In the **careful progress phase**, **TCP** will increase the congestion window size **CWND linearly**
- This **phase of TCP** is called the **congestion avoidance phase**



Congestion Avoidance Phase

- The **congestion avoidance phase** ends when **TCP detects a packet loss** and **TCP** repeats from **step 2** after setting:
- **SSThresh = CWND/2**
- **Here, CWND** is the value of the **congestion window** at which the **current congestion** took place !
- So there are two phases –
 - 1) Slow Start Phase
 - 2) Congestion Avoidance Phase



Significance of - **SSthresh**

- It is a marker between the

(1) Presumed safe phase and

(2) The unknown territory

- **TCP** maintains records the *assumed* safe transmission rate:
- **Slow Start Threshold (SSThresh)** = the *assumed* safe transmission rate



TCP operates in two different *modes/phases*

(1) Slow Start Mode/Phase: $CWND < SSThresh$

- This is the **start up** mode of operation of TCP
- In this mode/phase, **TCP** is sending **below** the *assumed* transmission rate
- **TCP** will try to **reach the safe transmission rate as fast as possible**
- In this phase, **TCP** will **start** by transmitting **ONE** packet in an epoch
- **At each successfully transmission epoch, TCP will double the transmission rate (resulting in an exponential increase !!!)**

Increase/Decrease function used:

- Increase: $x(t+1) = 2 \times x(t) \rightarrow$ **To reach the threshold fast**
- Decrease: $x(t+1) = 0.5 \times x(t) \rightarrow$ **To sharply drop down**



TCP operates in two different *modes/phases*

(2) **Congestion Avoidance Phase:**

The **slow start phase ends** when **TCP** has reached the **transmission rate** has reached the rate that **TCP** *believed* to be safe (SSThresh)

Unknown zone – Move on slowly - until **detect congestion** [it would be a waste of the available capacity] → But be **careful**

So, increase the transmission rate *slower* → Increase by *one packet* at each **successfully transmission epoch**

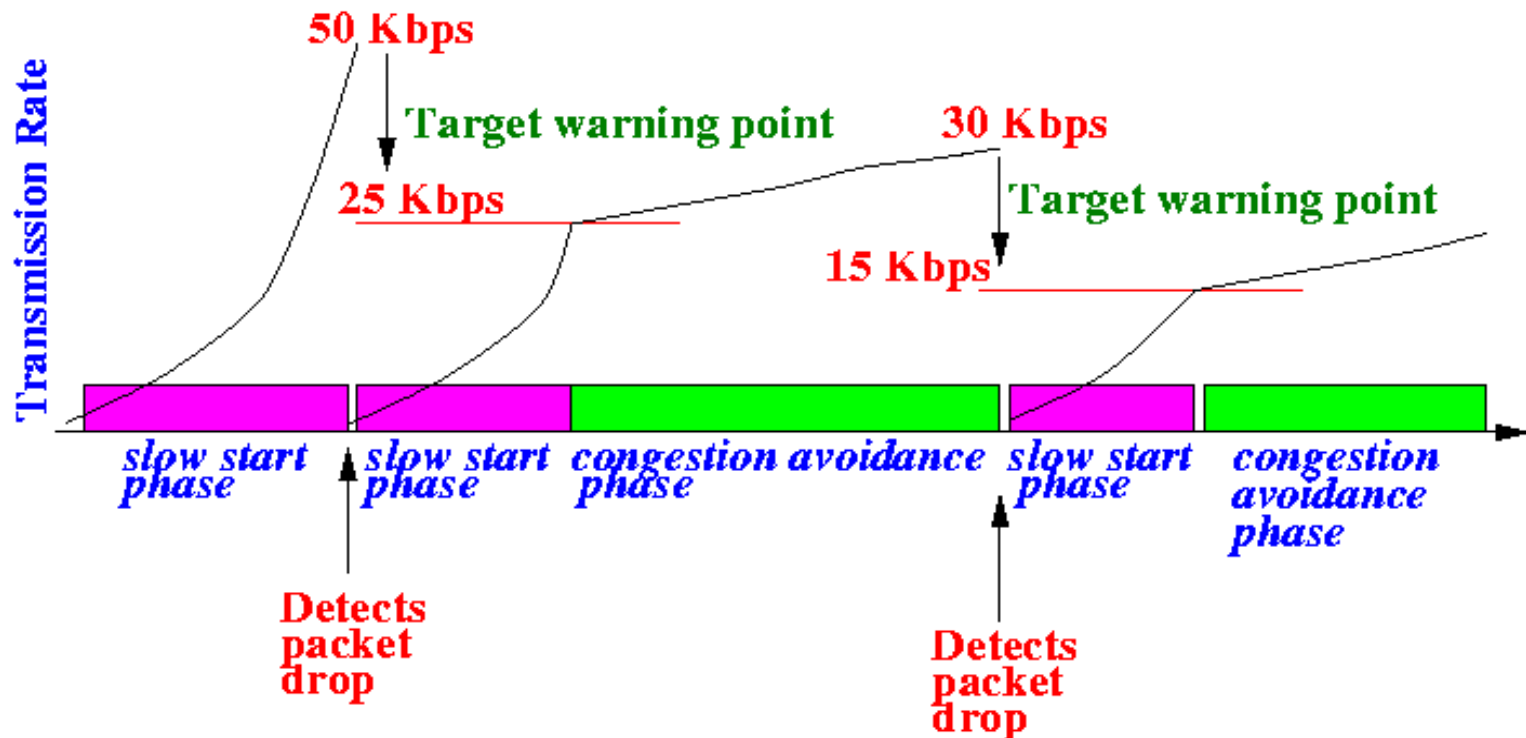
Increase/Decrease function used:

- Increase: $x(t+1) = x(t) + 1$ – **Slowly progress**
- Decrease: $x(t+1) = 0.5 \times x(t)$ - **Quickly drop down**



Graphical illustration of the TCP congestion control operation

- TCP transmission rate over time:
- (Note: bottleneck can change over time !!!)



How it starts [**When no idea of congestion**]

- When TCP starts out, it sets ***SSThresh = AWS***
- (So that it can try to send as much data as the receiver can handle)
- If the network can handle this transmission rate, TCP will not need to do any congestion control !!!
(Because the bottle neck is at the receiver NOT the network...)
- Recall our assumption:
- **AWS** is the **not the limiting factor**
- First graph shows a scenario where the **sender** starts losing packets at **50 Kbps**



When congestion is detected

- At some point (in the figure, it happens when sender was transmitting at **50 Kbps**),
Packets are dropped and **congestion is detected**.
- Because the packet drop happens at the moment when the sender was transmitting at **50 Kbps**
→ The **new target congestion rate** is set to:
SSThresh = 25 Kbps
- **TCP** now **restart** with **CWND = 1**
And **doubles** the **transmission rate** after **each epoch**
- The **transmission rate** will increase **exponentially** until it reaches **SSThresh = 25 Kbps**



Congestion avoidance...

- From **CWND = 25 Kbps** onwards, **TCP** will increase the **transmission rate *linearly*** - until it **detects a packet loss**
(In the figure, it happens when sender is transmitting **CWND = 30 Kbps**)
- Because the packet drop happens at the moment when the sender was transmitting **30 Kbps**, the new target congestion rate is set to: **SSThresh = 15 Kbps**
- **TCP** will again **restart** with **CWND = 1** and **doubles** the **transmission rate** after **each epoch**
- The **transmission rate** will increase ***exponentially*** until it reaches **SSThresh = 15 Kbps**



The process goes on...

- From **15 Kbps** onwards, TCP will increase the transmission rate **linearly** - until it discover a packet loss
- And so on....
- Imagine, the network state has changed –
- Now the congestion is supposed to start at 100 Kbps
- It will take now long time to reach that... from 15 to 100 linearly.
- **Situation keeps changing with time:**
- **So sometimes it behaves efficiently sometime wastes time...**



Remember

- One of the **goals** of TCP is to achieve the **highest possible throughput**.
- This goal is **not achieved** by **sending as fast as possible**, but **as fast as the *network can handle* !!!**
- The **available network bandwidth *changes continually*....**
- TCP tries to determine the available capacity by remembering when the data rate at which a packet drop occurred the **last time** and proceeds **carefully** starting from **half** way of this capacity level.
- When **TCP** starts transmitting for the ***first time***, it has **no idea** how much data the network can handle → So - **SSThresh = AWS**



So in summary

There are two phases -

- (1) In the **slow start phase**, TCP's transmission rate *doubles* after *each epoch* - thus increases **exponentially** in time
- (2) In the **congestion avoidance phase**, TCP's transmission rate increases *linearly* after each *epoch* - this increases **linearly** in time

